Unit-1

INTRODUCTION TO SOFTWARE QUALITY AND ASSURANCE

Prepared by: Ms. Aaska Bhatt

Index

- The software quality challenge
- What is Software quality?
- Software Quality Factors
- Overview of SQA components
- The SQA unit and other actors in the SQA System

Real-Life Problems

- How many of you have faced an app crash?
- How many of you saw a website showing '404 error'?
- Has your phone hung during a payment or an online exam?
- **<u>Definition</u>**
- **Software Testing = Finding errors before the customers find them.**
- or
- **Testing checks whether the software works the way it is expected to work.**

Why Testing Is Needed?

- **To avoid failures** (crashes, freezes, wrong results)
- **To save cost** (bugs are cheaper to fix early)
- **To ensure reliability** (users trust the software)

The software quality challenge

What the Warranty Basically Says

- **They are NOT guaranteeing anything** about the software's performance or quality.
- **They do NOT promise** that the software will do what you want it to do.
- **They do NOT guarantee** that the software or its documentation is free from errors (bugs).
- **If something goes wrong**, like:
  - Data loss
  - Loss of money
  - Problems caused by using the software
  - Claims from other people
    - **THEY ARE NOT RESPONSIBLE**
- The software is sold **"as is"** meaning: *use it at your own risk*.
- The **only thing they will replace** is:
  - The printed manual
  - The CD/DVD
  - But only if they are physically damaged and within 180 days of purchase

Why Are Software Warranties Like This?

- The answer: **software is very different from other products (like cars, machines, radios).**
- Here's why software quality assurance is unique:
- 1. Product Complexity
- 2. Product Visibility
- 3. Product development and production process
  - Product development
  - Product production planning
  - Manufacturing

Why Are Software Warranties Like This?

- **1. Product Complexity**
  - Machines may have a **few thousand possible operations**.
  - Software can have **millions of possible operations**.
  - It is impossible to test every single combination fully.

- More complexity → More chances of bugs.

Why Are Software Warranties Like This?

- **2. Product Visibility**
  - Hardware defects can be seen (e.g., missing button, broken part).
  - Software defects **cannot be seen** as it is stored inside CDs or digital files.
  - Missing components in software are also invisible.

- You can't visually inspect software like hardware.

Why Are Software Warranties Like This?

- **3. Product Development and product process**
  - For physical products (cars, washing machines), defects can be found in multiple stages:
    - **Design stage**
    - **Production planning**
    - **Manufacturing stage :** you can check and fix problems during mass production.
  - For software:
  - **Only the development stage** allows defect detection.
  - After that, manufacturing is just *copying* the software (CDs, downloads).
    No new defects can be detected there.
- Less opportunity to catch defects compared to hardware.

Software products vs. other industrial products

Characteristic	Software products	Other industrial products
Complexity	Usually, very complex product allowing for very large number of operational options	Degree of complexity much lower, allowing at most a few thousand operational options
Visibility of product	Invisible product, impossible to detect defects or omissions by sight (e.g. of a diskette or by sight CD storing the software)	Visible product, allowing effective detection of defects by sight
Nature of development and production process	Opportunities to detect defects arise in only one phase, namely product development	Opportunities to detect defects arise in all phases of development and production: ■ Product development ■ Product production planning ■ Manufacturing

Summary

- **High complexity**, as compared to other industrial products
- **Invisibility of the product**
- **Opportunities to detect defects ("bugs")** are limited to the product
- development phase

The environments for which SQA methods are developed

- **Software is developed by many types of people:**
  - **Students** making small projects
  - **Hobby coders** creating programs for fun
  - **Engineers, accountants, researchers** creating tools for their own work
  - **Professional software developers** building big products for companies and customers
- All of them face **software bugs**, but **professional developers** face the most serious quality issues.
- So, SQA (Software Quality Assurance) mainly focuses on **professional software development**, but its ideas can help everyone.

Why Professional Software Development Needs SQA?

- **The main characteristics of this environment are as follows:**

1. Contractual Conditions
2. Customer–Supplier Relationship
3. Need for Teamwork
4. Coordination With Other Teams
5. Interfaces With Other Software Systems
6. Team Members May Leave
7. Maintenance Is Needed for Many Years

Why Professional Software Development Needs SQA?

- **1. Contractual Conditions:**

- When a company develops software for a customer, a **contract** decides:
  - What features the software must have
  - How much money will be spent (budget)
  - When the project must finish (timetable)

- So developers must follow all requirements strictly.

Why Professional Software Development Needs SQA?

- **2. Customer–Supplier Relationship**

- During development, the customer constantly:
  - Requests changes
  - Gives feedback
  - Reviews progress
  - Approves modifications

- So there is continuous communication.
- This does not happen in student or hobby projects.

Why Professional Software Development Needs SQA?

- **3. <u>Need for Teamwork</u>**

- Software projects need a **team**, not just one person, because:
  - Workload is large
  - Different skills are required (analysis, coding, testing, design)
  - Team members can help each other and improve quality

Why Professional Software Development Needs SQA?

- **4. Coordination With Other Teams**

- Large projects often require working with:
  - Other software teams
  - Hardware teams
  - Teams from other companies
  - Customer's own teams

- Everyone must coordinate so the whole system works together.

Why Professional Software Development Needs SQA?

- **5. Interfaces With Other Software Systems**

- Most software today needs to work with other software.

- Example: **Salary processing software**
  - **Input**: Attendance data from time-clock software
  - **Output**: Salary transfer data sent to bank software

- These connections are called **interfaces**.
- If an interface fails, the entire system may break.

Why Professional Software Development Needs SQA?

- **<u>6. Team Members May Leave</u>**
- In long projects, sometimes developers leave the company.

- The team leader must:
  - Add a new person
  - Train them
  - Continue the project without delays

- The work cannot stop because the timeline is fixed.

Why Professional Software Development Needs SQA?

- **7. Maintenance Is Needed for Many Years**

- After software is delivered, it must be maintained for:
  - Bug fixing
  - Adding new features
  - Updates

- Customers expect support for **5–10 years**.

- So SQA must ensure long-term quality.

Why Professional Software Development Needs SQA?

- **<u>In Simple Words:</u>**
- Professional software development is complex because:
  - It involves contracts
  - Customers are involved throughout
  - Large teams must work together
  - Systems depend on each other
  - Projects run for a long time
  - People may join or leave
  - Maintenance continues for years
- This environment requires **strong SQA methods** to avoid failures.

What is Software?

- **1. Software is more than just code**
  - People usually think software = programming instructions.
  - But code alone is **not enough** to ensure quality.
- **2. IEEE definition of software includes 4 components**
  - **Software is:** Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.
  - According to IEEE (1991), software consists of:
    - **Computer programs (code)**
    - **Procedures**
    - **Documentation**
    - **Data required to operate the system**

What is Software?

- **<u>3. Why are all four components needed for quality?</u>**
- **a. Computer Programs (Code)**
  - They make the computer perform the required tasks.
- **b. Procedures**

  They specify:
  - Order of operations
  - How programs are executed
  - Who is responsible
- Needed for smooth and correct execution.

What is Software?

- **c. Documentation**
- There are different types:
  - **Development documentation**
    Helps developers coordinate, review, and inspect the system.
  - **User documentation**
    Helps users understand how to use the software.
  - **Maintenance documentation**
    Helps maintenance team fix bugs, understand structure, and update software.
- **d. Data**
  - Includes parameters, codes, settings, lists, etc.
  - Adapts software to specific user needs.
  - Standard test data is required to detect unexpected changes or errors.

Software Errors, Faults & Failures

- "How can the same software be good, bad, and suddenly bad again?"
- 1. **What causes software failures?**
- 2. **Not every error becomes a fault**
- 3. **Not every fault becomes a failure**
- **1. What causes software failures?**
  - Failures come from **errors (mistakes)** made by the programmer.
  - These errors can be:
    - Grammatical mistakes in code
    - Logical mistakes (wrong requirement implementation)

Software Errors, Faults & Failures

- **2. Not every error becomes a fault**
  - Imagine you typed the wrong spelling in a WhatsApp message.
    Sometimes autocorrect fixes it → no issue
    Sometimes your friend still understands → no issue
    Sometimes it creates confusion → issue occurs
- **Same way:**
  - Some errors → **no impact**
  - Some errors → **cause faults**
  - Some errors → **get neutralized** by other code

Software Errors, Faults & Failures

- **3. Not every fault becomes a failure**
- *"If there's a bug in a feature that you NEVER use, will you see a failure?"*
- A fault becomes a **failure only when it gets activated**, meaning:
  - The user performs the faulty action
  - OR the exact combination of conditions occurs
- **Example:**
  There is a bug in the "Export to PDF" feature.
  You never use it → **No failure**
  Someone else uses it daily → **Failure every day**

Software Errors, Faults & Failures

- **Example 1 – "Meteoro-X" Meteorological Equipment Firmware**
- The **Meteoro-X** device had a requirement:
  → *Stop working automatically if internal temperature goes above **60°C*** to prevent damage.
- The programmer made a **mistake** (software error):
  → He coded the limit as **160°C** instead of 60°C.
  → This created a **software fault**.
- **Why did this fault NOT become a failure?**
  → Because the equipment was used **only in coastal areas**, where temperature **never reaches 60°C**.
  → So even though the fault existed, it was **never activated**.
- **Conclusion:**
  - Many software faults remain **hidden** because the situation needed to trigger them never occurs.
  - Users only notice **failures**, not errors or faults.
  - Developers care about **errors + faults**, users care about **failures**.

Software Errors, Faults & Failures

- **Definitions:**
- **Software errors** are sections of the code that are partially or totally incorrect as a result of a grammatical, logical or other mistake made by a systems analyst, a programmer, or another member of the software development team.
- **Software faults** are software errors that cause the incorrect functioning of the software during a specific application.
- **Software faults** become **software failures** only when they are "activated", that is, when a user tries to apply the specific software section that is faulty. Thus, the root of any **software failure** is a **software error.**

Classification of the causes of Software Errors

- 1. Faulty definition of requirements
- 2. Client-developer communication failures
- 3. Deliberate deviations from software requirements
- 4. Logical design errors
- 5. Coding errors
- 6. Non-compliance with documentation and coding instructions
- 7. Shortcomings of the testing process
- 8. Procedure errors
- 9. Documentation errors

Classification of the causes of Software Errors

- **1. Faulty definition of requirements**

1. <u>Wrong Requirements (Erroneous Definition)</u>

- The client gives **incorrect information** about what the software should do.
  Example: A billing system requirement says tax = 15%, but actual tax is 18%.

2. <u>Missing Important Requirements (Absence of Vital Requirements)</u>

- Something very important is **not mentioned at all**.
  Example: A hospital system forgot to include "emergency patient registration".

Classification of the causes of Software Errors

- **1. Faulty definition of requirements**

  - **<u>3. Incomplete Requirements</u>**
  - Some parts are written; some parts are **left out**.
    Example from the passage:
    A municipal tax software includes discounts for:
    - Senior citizens
    - Large families
      …but **forgot** to include a discount for **students**.
- This missing detail becomes a **major software problem later**.

Classification of the causes of Software Errors

- **1. Faulty definition of requirements**

- **<u>4.Unnecessary Requirements</u>**
  - Clients sometimes add features that are **not needed now** and may never be used.
    This:
  - Wastes time
  - Makes software more complicated
  - Can create more errors
- Example: Asking a small library system to include a "multilingual AI chatbot" which is not needed.
- In short: If requirements are wrong, missing, incomplete, or unnecessary → software will have more **errors, faults, and failures**.

Classification of the causes of Software Errors

- **2. Client – developer communication failures**
- Even if requirements are written correctly, **poor communication** between the client and developer can create software errors.
- Most of these misunderstandings happen in the **early stages** and lead to wrong features or missing features.
- **1. Misunderstanding the Requirement Document**
  - Developer reads the document but interprets something **incorrectly**.
  - Example:
    Client says "generate monthly report," but developer creates "monthly + weekly report".

Classification of the causes of Software Errors

- **2. Client – developer communication failures**

- **2. Misunderstanding Written Changes**
  - Clients sometimes send changes through emails or documents.
    If the developer misreads or misunderstands them → errors occur.
  - Example:
    Client writes "Increase limit to 50 users" but developer thinks "50 employees".
- **3. Misunderstanding Oral Changes**
  - Changes told **verbally** are the riskiest.
  - Example:
    Client says in a meeting: "Add priority customers", and developer thinks only "Platinum customers", but client meant "Platinum + Gold".

Classification of the causes of Software Errors

- **2. Client – developer communication failures**
- **4. Misunderstanding Client's Answers**
  - If the developer asks a question and **misinterprets** the client's answer, wrong implementation happens.
  - Example:
    Developer: "Should the discount apply to all categories?"
    Client: "Yes, generally."
    Developer thinks "everyone", but client only meant "regular customers, not new ones".
- **5. Ignoring Client Communication**
- Sometimes developers:
  - Skip reading emails
  - Miss updates
  - Don't pay attention to requirement changes
- This also causes errors.

Bad communication → wrong understanding → wrong development → software errors.

Classification of the causes of Software Errors

- **<u>3. Deliberate deviations from software requirements:</u>**
  - Sometimes developers **intentionally** do something *different* from what the requirement document says.
    Their intention may be good or practical, but these deviations often create **new software errors**.
- **<u>1. Reusing Old Code Without Proper Checking</u>**
  - Developers sometimes copy modules from an old project to save time.
  - But if they don't analyze:
  - what needs to be changed
  - what doesn't match the new project
  - …then errors appear.
- **Example:**
  Using a billing module from a *shopping app* in a *hospital system*.
  Without modification, it may calculate discounts incorrectly.

Classification of the causes of Software Errors

- **3. Deliberate deviations from software requirements:**

- **2. Skipping Some Features Due to Time or Budget Pressure**
- Deadlines and budgets push developers to deliver faster.
- So they may quietly remove or delay:
  - some validations
  - some reports
  - rare-use features
- This causes missing requirements → errors later.
- **Example:**
  Skipping "password reset via SMS" because "we'll add it later".

Classification of the causes of Software Errors

- **3. Deliberate deviations from software requirements:**

- **3. Developer Adds "Improvements" Without Asking the Client**
- Sometimes developers add features they think are "better", but without client approval.
- They may think it's a small change, but it can break the system.
- **Example:**
  Developer adds "auto-sorting" in a customer list →
  Client complains because their work requires **original data order**.
- **In short:**
- Even with good intentions, changing requirements without analysis or approval →
  creates software errors.

Classification of the causes of Software Errors

- **<u>4. Logical errors:</u>**
- Software errors can enter the system when the professionals who design the system – systems architects, software engineers, analysts, etc. – formulate the software requirements.
- **Types of Logical errors:**
- **1. Wrong Algorithms (Incorrect Logic Design)**
- The requirement is correct, but the analyst writes the wrong logic during design.
- **Example:**
- Requirement: "A discount applies to customers who purchase **more than 3 times** in a month."
- Analyst defines algorithm:
  "Discount applies to customers who purchase **3 times or more**."
- → Logical mistake → system will give discount to wrong customers

Classification of the causes of Software Errors

- **<u>4. Logical errors:</u>**

- **2. Sequencing Errors (Wrong Order of Steps)**
- The order in which tasks must be executed is misunderstood or incorrectly documented.
- **Example (From the book):**
- Debt collection process:
  3 reminder letters → Sales Manager decides → Legal Department
- Analyst defines:
  3 letters → Directly Legal Department
  → Sales step missing → Wrong process flow

Classification of the causes of Software Errors

- **<u>4. Logical errors:</u>**

- **3. Wrong Boundary Conditions**
- Analyst incorrectly defines limits or threshold values.
- **Example:**
- **Requirement:** "Special discount for customers with **more than 3 purchases**."
  Analyst writes: "Discount for customers with **3 or more purchases**."
- → Wrong boundary → Causes incorrect discounting.

Classification of the causes of Software Errors

- **<u>4. Logical errors:</u>**

- **4. Missing System States (Forgotten Conditions)**
- Analyst forgets to define how the system should behave under certain conditions.
- **Example:**
- A real-time device must react when:
  Temperature > 120°C AND Pressure is 6–8 atm
  Analyst forgets to define this combination.
- → System doesn't know what to do → error or failure.

Classification of the causes of Software Errors

- **<u>4. Logical errors:</u>**

- **5. Missing Handling for Illegal Operations (Invalid Input Cases)**
- Requirement says how the system should behave, but analyst doesn't define the software reaction for invalid user actions.
- **Example (From the book):**
- **Requirement:** "Maximum 10 tickets per customer."
  Analyst shows the message "Only 10 tickets allowed"
  but does NOT define what happens if user enters 20.
- → System may crash or freeze because reaction is not defined.

Classification of the causes of Software Errors

- **5. Coding errors:**

- **1.** Misunderstanding the design documentation
- 2. Language syntax errors (Linguistic errors)
- 3. Wrong use of CASE tools
- 4. Errors in data selection or data handling
- 5. Logical mistakes inside the code
- 6. Typing mistakes (simple human errors)
- 7. Copy–paste errors

Classification of the causes of Software Errors

- **<u>6. Non-compliance with documentation and coding instructions</u>**
- When a developer **does not follow documentation or coding standards**, it doesn't only affect their own work — it affects the **entire team**.
- Team members struggle to coordinate their code
- Replacement of that developer becomes a nightmare
- Reviewers face difficulty
- Testing becomes harder and less efficient
- Maintenance becomes extremely risky

Classification of the causes of Software Errors

- **7. Shortcomings of the testing process**
- If the testing process is weak or incomplete, many errors remain **undetected** or **uncorrected**. This increases the overall error rate in the software.
  - Incomplete Test Plans
  - Errors Are Not Properly Reported
  - Wrong Understanding of the Bug
  - Incomplete Fixes

Classification of the causes of Software Errors

- **<u>8. Procedure errors</u>**
- **Procedures** are step-by-step instructions that tell the user *how* to use the software correctly.
  - Work is done in multiple steps
  - Different types of data are processed
  - Intermediate results need to be checked
  - The final result depends on following the correct sequence
- If procedures are not followed correctly → wrong results, wrong bills, wrong outputs, and confusion.

Classification of the causes of Software Errors

- **9. Documentation errors**
- Documentation is not only for users — it is also for developers and maintenance teams.
  When documentation contains mistakes, it creates **confusion, misuse, extra bugs, and wrong fixes**.
- There are **two major types** of documentation errors:
- **1.** Documentation Errors that Affect Developers & Maintenance Teams
- **2.** Documentation Errors that Affect Users
  - Omission of software functions
  - Wrong
  - Listing non-existing functionsexplanations or wrong instructions

What is Software Quality?

Software quality - Definition

- **Software quality – IEEE definition**
- **Software quality is:**
- 1. The degree to which a system, component, or process meets specified requirements.
- 2. The degree to which a system, component, or process meets customer or user needs or expectations.

"Quality means conformance to requirements"

"(1) Quality consists of those product features which meet the needs of customers and thereby provide product satisfaction.

(2) Quality consists of freedom from deficiencies"

Software quality - Definition

- **Software quality – IEEE definition**

- 1. The degree to which a system, component, or process meets specified requirements.
- 2. The degree to which a system, component, or process meets customer or user needs or expectations.

"Quality means conformance to requirements"

"(1) Quality consists of those product features which meet the needs of customers and thereby provide product satisfaction.

(2) Quality consists of freedom from deficiencies"

Software quality - Definition

"Quality means conformance to requirements" – (Crosby)

"(1) Quality consists of those product features which meet the needs of customers and thereby provide product satisfaction.

(2) Quality consists of freedom from deficiencies" (Juran)

- **Crosby's definition –** Errors included in the software specification are not considered and do not reduce the software quality, a characteristics that can be considered the approach's deficiency.
- **Juran's definition –** Achieving customer satisfaction and views the fulfillment of customer's real needs as the true goal of software quality.
- Adopting the second definition demands that the developer invest significant

professional efforts in examining and correcting, if necessary, the customer's requirements specifications.

Software quality assurance – definition and objectives

- **Software quality assurance – IEEE definition**
- **Software quality assurance is :**
- 1. A planned and systematic pattern of all actions necessary to provide adequate confidence that an item or product conforms to established technical requirements.
- 2. A set of activities designed to evaluate the process by which the products are developed or manufactured. Contrast with quality control.
- This definition may be characterized in the following:
  - SQA ensures a planned, systematic set of activities across all development stages to assure the client that the software will meet all technical requirements.
  - Refer to the software development process.
  - Refer to the specifications of the technical requirements.

Software quality assurance vs software quality control

- Quality control involves evaluating a finished product to ensure it meets required standards and withholding any product that does not qualify before it is delivered to the client. In contrast, **quality assurance** is meant to minimize the costs of quality by introducing a variety of activities throughout the development and maintenance process in order to prevent the causes of errors, detect them, and correct them in the early stages of development. As a result, quality assurance substantially reduces the rates of non-qualifying products.
- **In sum:**

(1) Quality control and quality assurance activities serve different objectives.

(2) Quality control activities are only a part of the total range of quality assurance activities.

Objectives of Software Quality Assurance

- **The objectives of SQA activities for software development and maintenance are:**

(1) Assuring, with acceptable levels of confidence, conformance to functional technical requirements.

(2) Assuring, with acceptable levels of confidence, conformance to managerial requirements of scheduling and budgets.

(3) Initiating and managing activities for the improvement and greater efficiency of software development and SQA activities.

Relationship between Software Quality Assurance and Software Engineering

- **Software engineering** is the application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software. The characteristics of software engineering, especially its systematic, disciplined and quantitative approach, make software engineering a good environment for achieving SQA objectives. It is commonly accepted that cooperation between software engineers and the SQA team is the way to achieve efficient and economic development and maintenance activities that, at the same time, assure the quality of the products of these activities.

Software Quality Factors

The need for a comprehensive definition of requirements

- **There are some characteristic common to all these "but's":**
- All the software projects satisfactorily fulfilled the basic requirements for correct calculations (correct inventory figures, correct average class's score, correct loan interest, etc.).
- All the software projects suffered from poor performance in important areas such as maintenance, reliability, software reuse, or training.
- The cause for the poor performance of the developed software projects in these areas was the lack of predefined requirements to cover these important aspects of the software's functionality.

The need for a comprehensive definition of requirements

- **1. Comprehensive Requirements:**
  Software requirements must cover all attributes of the software such as usability, reusability, maintainability, performance, and other aspects to ensure complete user satisfaction.
- **2. Need for Quality Factors:**
  Since software requirements include many different issues, they are grouped into **quality factors** for better organization and understanding.
- **3. Role of Requirement Team:**
  The team preparing the Software Requirements Document (SRD) must examine each quality factor and decide whether requirements related to that factor are needed.

Software Quality Factors

- **<u>The need for a comprehensive definition of requirements</u>**
- **4. Project-Specific Emphasis:**
  Not all software projects require the same quality factors. Therefore, requirement documents differ based on the nature and priorities of each project.
- **5. Variation Across Documents:**
  Because project needs vary, some quality factors may be emphasized more, while others may not appear at all in certain requirement documents.
- **6. Classification of Quality Requirements:**
  The following sections classify these quality requirements into major quality factors, presenting only the widely accepted approaches.

Classifications of software requirements into software quality factors

- McCall's factor model classifies all software requirements into 11 software quality factors. The 11 factors are grouped into three categories – product operation, product revision and product transition – as follows:
- **1. Product operation factors:** Correctness, Reliability, Efficiency, Integrity, Usability.
- **2. Product revision factors:** Maintainability, Flexibility, Testability.
- **3. Product transition factors:** Portability, Reusability, Interoperability.

Product operation software quality factors

- **Correctness:** Correctness means that the software produces **accurate, complete, and reliable outputs exactly as required** by the system specifications.
- Example:
- A club membership system defines correctness as:
- Must produce 11 reports, 4 letters, and 8 queries (output mission).
- Wrong outputs must be less than 1% (accuracy).
- Missing member details must be less than 1% (completeness).
- Updates must be done within 1–2 working days (up-to-date ness).
- Queries in <2 seconds, reports in <4 hours (availability).

Product operation software quality factors

- **Reliability:** Reliability means **how often the software fails** and **how long it stays unavailable**. It defines the **maximum allowed failure rate** for the system or specific functions.
- **Examples**
- **Example 1 (Medical System)**
- A heart-monitoring unit must fail **less than once in 20 years**, and its heart-attack detection feature must fail **less than once in 1,000,000 cases**.
- **Example 2 (Banking System)**
- A bank's software should not be down for more than **10 minutes per month** during working hours, and the chance of repair time taking **over 30 minutes** must be **less than 0.5%**.
- Queries in <2 seconds, reports in <4 hours (availability).
- Must follow client's coding/documentation standards.

Product operation software quality factors

- **Efficiency:** Efficiency means **how well the software uses hardware resources** like **processor speed, memory, storage, and network bandwidth**. A system is efficient if it performs all required functions **using minimum hardware and power**.
- **Examples**
- **Example 1 (Store Chain System)**
- Two software bids use the same computers but require different hardware:
- **Bid B** uses **less storage** and **fewer communication lines** than Bid A. Therefore, **Bid B is more efficient** because it needs fewer hardware resources.
- **Example 2 (Meteorological Unit)**
- A weather-monitoring device with a **1000 mAh battery** must run for **30 days**, taking readings hourly and sending data once per day. This requires **efficient power usage**.

Product operation software quality factors

- **Integrity**
- Integrity means **protecting the software and data from unauthorized access or changes**. It ensures that only authorized people can **read** or **modify** information.

- **Example**
- A municipality's GIS system allows citizens to **view and copy maps** but **not edit them**. Some sensitive maps (like plans in progress) are **restricted** and cannot be accessed.

Product operation software quality factors

- **Usability**
- Usability refers to **how easy the software is to learn and use**, including how much training staff need and how efficiently they can operate the system.

- **Example**
- For a home-appliance service company's help desk system:
- A staff member should handle **60 calls per day**.
- A new employee should be trained in **2 days** and then handle **45 calls per day**.

Product revision software quality factors

- These factors support:
- **Corrective maintenance** – fixing bugs and failures
- **Adaptive maintenance** – adjusting software for new environments/customers
- **Perfective maintenance** – improving or enhancing existing features
- These three quality factors ensure that the software can be **changed, updated, or improved efficiently**.

Product revision software quality factors

- **Maintainability**
- Maintainability refers to how **easily and quickly** the software can be **understood, fixed, or updated** when it fails or needs improvement. It depends on things like good modular design, clear documentation, and clean coding standards.
- **Example**
- Each software module should contain **no more than 30 statements**.
- Code must follow **company coding standards** for easier understanding.

Product revision software quality factors

- **Flexibility**
- Flexibility refers to how **easily the software can be adapted** for new customers, new conditions, new reports, or changes in the environment.
  It shows how much effort (time, manpower) is needed to modify or extend the software without major rework.
- **Example**
- The software must work for **all teachers**, all subjects, and all school levels.
- Even **non-technical staff** should be able to create new report formats as needed.

Product revision software quality factors

- **Testability:** Testability refers to how **easy it is to test** the software to find errors. Good testability means the system has built-in features that help testers, such as:
  - Log files
  - Intermediate results
  - Automatic diagnostic checks
- It also includes self-tests performed before the system starts, to ensure all components are working properly.
- **Example**
- A computerized industrial control unit uses **standard test data** every morning before production starts.
  The system's expected outputs are known, so running this test ensures the unit is working correctly.

Product transition software quality factors

- **Portability**
- Portability refers to how easily a software system can be **moved or adapted** to **different hardware**, **operating systems**, or **environments**.
  High portability means the same software can run in multiple environments with **minimum changes and cost**.

- **Example**
- A software package created for **Windows 2000** should be easily transferable to **Linux** or **Windows NT** with low cost and minimal modification.

Product transition software quality factors

- **Reusability:**
- Reusability means designing software modules in such a way that they can be **used again in future projects**. It helps to **save time, reduce cost, shorten development**, and improve quality because reused modules are already tested and reliable.
- **Example:**
- A team developing a **hotel swimming pool system** designs some modules (like membership card checking, restaurant billing, renewal processing) so they can be **reused later** for the **hotel spa system** planned next year.

Product transition software quality factors

- **Interoperability**
- Interoperability ensures that a software system or firmware can **work together with other software or equipment**, following **standard interfaces or data structures**.

- **Example**
- Medical lab equipment firmware outputs results in a **standard format** so they can be **used by multiple laboratory information systems**.

Alternative models of software quality factors

- **Verifiability**
  - Ensures **efficient verification** of design and code.
  - Focus on **modularity, simplicity, documentation, coding standards**.
- **Expandability**
  - Supports future growth: **larger user base, new applications, improved service**.
  - Related to McCall's **Flexibility** factor.
- **Safety**
  - Prevents **hazardous conditions** for equipment operators due to software errors.
  - **Example**: Chemical plant system controlling acid flow; alarms and reactions defined for dangerous situations.

Alternative models of software quality factors

- **Manageability**
  - Tools for **software modification and administration** during development/maintenance.
  - **Example**: “Chemilog” chemical tracking system controlled by software change procedures.

- **Survivability**
  - Ensures **continuity of service** defines minimum time between failures and maximum recovery time.
  - **Example**: Taya National Lottery – probability of unrecoverable data loss < 1 in a million.

Overview of the Software quality assurance system

The SQA system – an SQA architecture

- **SQA system components can be classified into six classes:**
- 1. Pre-project components
- 2. Software project life cycle components
- 3. Infrastructure components for error prevention and improvement
- 4. Management SQA components
- 5. SQA standards, system certification and assessment components
- 6. Organizing for SQA – the human components

The SQA system – an SQA architecture

- **1. Pre-project components:**
  - Contract Review
  - Development & Quality Plans
- **4.2.1 Contract Review**
  - Before starting the project, all documents and requirements are checked carefully.
- **Activities included:**
  - Clarifying customer requirements
  - Reviewing project schedule & resource estimates
  - Checking whether the team has the capability to do the project
  - Checking customer's ability to meet their responsibilities
  - Evaluating development risks
  - For maintenance contracts: check scope (error correction, adaptation, small enhancements)

The SQA system – an SQA architecture

- **1. Pre-project components:**
- **Development and Quality Plans:** After the contract is signed, two plans are prepared:
  - **Development Plan**
  - **Quality Plan**
- **Development Plan includes:**
  - Project schedule and timeline
  - Required manpower & hardware
  - Risk evaluation
  - Organizational structure (teams, subcontractors)
  - Development methodology & tools
  - Software reuse plans
- **Quality Plan includes:**
  - Quality goals (measurable targets)
  - Entry & exit criteria for each project stage
  - List of reviews, tests, verification & validation activities

The SQA system – an SQA architecture

- **2. Software project life cycle components:**
- The software project life cycle has **two major stages**:
  - **Development Life Cycle**
  - **Operation–Maintenance Stage**
- Main SQA components used in the life cycle:
  - Reviews
  - Expert opinions
  - Software testing
  - Software maintenance
  - Quality assurance of subcontractors & customer-supplied parts

The SQA system – an SQA architecture

- **2. Software project life cycle components:**
- **Reviews**
- **Types of Reviews**
  - **Formal Design Reviews (DRs)**
  - **Peer Reviews** (inspections, walkthroughs)
- **Formal Design Reviews (DRs)**
- Conducted for major design documents (design reports, test plans, manuals).
- Required before moving to next phase.
- Done by senior professionals: project leader, manager, chief engineer, etc.
- Customer representative may join.
- DR outcomes:
  - Immediate approval
  - Conditional approval (after corrections)
  - Require another DR after fixes
- Output: list of **action items** (required corrections)

The SQA system – an SQA architecture

- **2. Software project life cycle components:**
- **Reviews**
- **Types of Reviews**
  - **Formal Design Reviews (DRs)**
  - **Peer Reviews** (inspections, walkthroughs)
- **Peer Reviews**
  - For small documents, code modules, sections of reports.
  - Done by colleagues/peers (not seniors).
  - Aim: detect design/programming faults early.
  - Output: list of defects; inspections also produce statistics for improvement.
  - Based on reciprocity (colleagues help each other).

The SQA system – an SQA architecture

- **2. Software project life cycle components:**
- **Expert Opinions**
- Used when internal capability is insufficient.
- **Situations requiring expert opinions:**
  - Lack of expertise inside the organization
  - Small organizations lacking experienced reviewers
  - When staff is overloaded or unavailable
  - To resolve major disagreements among senior staff
  - To replace DR or inspection when necessary

The SQA system – an SQA architecture

- **2. Software project life cycle components:**
- **Software Testing**
  - Formal SQA activity to test the actual running software.
  - Based on pre-defined **test cases** for different scenarios.
  - Includes module tests, integration tests, system tests.
  - **Regression testing** done after corrections.
  - Purpose: detect faults + give approval to move to next programming phase or delivery.
  - Tests may be manual or automated.
  - Test report includes list of faults + recommendation for re-tests.
  - Best practice: tests should be done by **independent testing team**, not developers.

The SQA system – an SQA architecture

- **3. Infrastructure components for error prevention and improvement**
- This class of SQA components includes:
  - Procedures and work instructions
  - Templates and checklists
  - Staff training, retraining, and certification
  - Preventive and corrective actions
  - Configuration management
  - Documentation control.

The SQA system – an SQA architecture

- **3. Infrastructure components for error prevention and improvement**
- 1. Procedures and work instructions
- **Procedures**
  - Provide **detailed guidelines** for performing specific development activities.
  - Are **general** and applicable to the **entire organization**.
  - Ensure that processes are performed in a **standard, quality-controlled** manner.
- **Work Instructions**
  - Give **step-by-step directions** for tasks used in **specific situations**.
  - Usually prepared for **specialized teams** or unique activities.

The SQA system – an SQA architecture

- **3. Infrastructure components for error prevention and improvement**
- 2. Supporting Quality devices
- **Purpose / Benefits**
- **Save time**
  – No need to design document formats or create review checklists from scratch.
- **Ensure completeness**
  – Help make sure all required points are covered in documents and reviews.
- **Improve communication**
  – Standardized formats make it easier for development teams and review committees to understand each other.
- **Why They're Useful**
- Based on the organization's **experience and knowledge**.
- Promote **consistency**, **accuracy**, and **faster review**.

The SQA system – an SQA architecture

- **3. Infrastructure components for error prevention and improvement**
- 3. Staff training, instruction and certification
- **Training New and Reassigned Employees**
  - New employees receive basic and role-specific training.
  - Employees shifting to new roles are retrained to meet updated responsibilities.
- **Continuous Professional Updates**
  - Staff are regularly updated about new technologies, tools, methods, and industry practices.
  - Includes sharing in-house experience, lessons learned, and best practices.
- **Certification of Employees**
  - Employees are certified only after demonstrating required skills and knowledge.
  - Ensures the workforce meets organizational quality standards.

The SQA system – an SQA architecture

- **4. Management SQA components**

  Control components include:

  - 1. Project progress control (including maintenance contract control)
  - 2. Software quality metrics
  - 3. Software quality costs.

- **1. Project progress control**
  - The main objective of project progress control is to **identify deviations from project plans and maintenance performance early**, so that **timely and effective corrective actions** can be taken.

  Project control activities focus on:

  - Resource usage
  - Schedules
  - Risk management activities
  - The budget.

The SQA system – an SQA architecture

- **4. Management SQA components**
  - 1. Project progress control (including maintenance contract control)
  - 2. Software quality metrics
  - 3. Software quality costs.
- **2. Software quality metrices**
  - Software quality metrics help measure different aspects of software performance and development processes. They support **control**, **decision-making**, and **continuous improvement** during development and maintenance.

Quality of software development and maintenance activities

- Development teams' productivity
- Help desk and maintenance teams' productivity
- Software faults density
- Schedule deviations.

The SQA system – an SQA architecture

- **5. SQA standards, system certification, and assessment components**
- **The main objectives of this class of components are:**
- (1) Utilization of international professional knowledge.
- (2) Improvement of coordination with other organizations' quality systems.
- (3) Objective professional evaluation and measurement of the achievements of the organization's quality systems.

The SQA system – an SQA architecture

- **5. SQA standards, system certification, and assessment components**
- The standards available may be classified into two main sub-classes: quality management standards and

  project process standards.

1. Quality management standards

Management-oriented SQA standards define **what quality requirements must be met**, not **how to meet them**, allowing organizations flexibility.

- The most familiar examples of this type of standard are:
  - SEI CMM assessment standard
  - ISO 9001 and ISO 9000-3 standards.

The SQA system – an SQA architecture

- **5. SQA standards, system certification, and assessment components**
- The standards available may be classified into two main sub-classes: quality management standards and

  project process standards.

2. Project process standards

- Project process standards are professional standards that provide methodological guidelines (dealing with the question of "how") for the development team. Well-known examples of this type of standards are:
  - IEEE 1012 standard
  - ISO/IEC 12207 standard.

The SQA system – an SQA architecture

- **6. Organizing for SQA – the human components**

- The main objectives of the SQA organizational base are as follows:
  - To develop and support implementation of SQA components.
  - To detect deviations from SQA procedures and methodology.
  - To suggest improvements to SQA components.
  - SQA organizational base concentrates on specific tasks.
    - **1. Management's role in SQA**
    - **2. The SQA Unit**
    - **3. SQA trustees, committees and forums**

The SQA system – an SQA architecture

- **6. Organizing for SQA – the human components**
- **1. Management's role in SQA**
- The responsibilities of top management (through the executive in charge of software quality), departmental management and project management include the following:
  - **Definition** of the quality policy
  - Effective **follow-up** of quality policy implementation
  - Allocation of sufficient resources to implement quality policy
  - Assignment of adequate staff
  - Follow-up of compliance of quality assurance procedures
  - Solutions of schedule, budget and customer relations difficulties.

The SQA system – an SQA architecture

- **6. Organizing for SQA – the human components**
- **2. The SQA Unit**
- The SQA unit and software testers work **full-time** on software quality, while other staff contribute **part-time**. Therefore, the SQA unit acts as the **central driving force** of the SQA system, serving as its **initiator, coordinator, and controller**, and is responsible for carrying out key quality assurance roles across the organization.
- **This task can be broken down into a number of primary roles:**
- Preparation of annual quality programs
- Consultation with in-house staff and outside experts on software quality issues
- Conduct of internal quality assurance audits
- Leadership of quality assurance various committees

The SQA system – an SQA architecture

- **6. Organizing for SQA – the human components**
- **3. SQA trustees, committees and forums**
- **Their contributions include:**
  - Solving team or unit local quality problems
  - Initiating improvements in SQA components
  - Reporting to the SQA unit about quality issues in their team or unit.
  - Solution of software quality problems.
  - Analysis of problem and failure records as well as other records, followed by initiation of corrective and preventive actions when appropriate.
  - Initiation and development of new procedures and instructions; updating existing materials.

The SQA unit and other actors in the SQA System

The SQA system – an SQA architecture

- SQA activities represent all or part of their standard assignments or whose participation in SQA bodies goes beyond their regular activities, namely:
  - 1. SQA unit members
  - 2. SQA trustees
  - 3. SQA committee members
  - 4. SQA forum members

The SQA system – an SQA architecture

1. SQA unit members

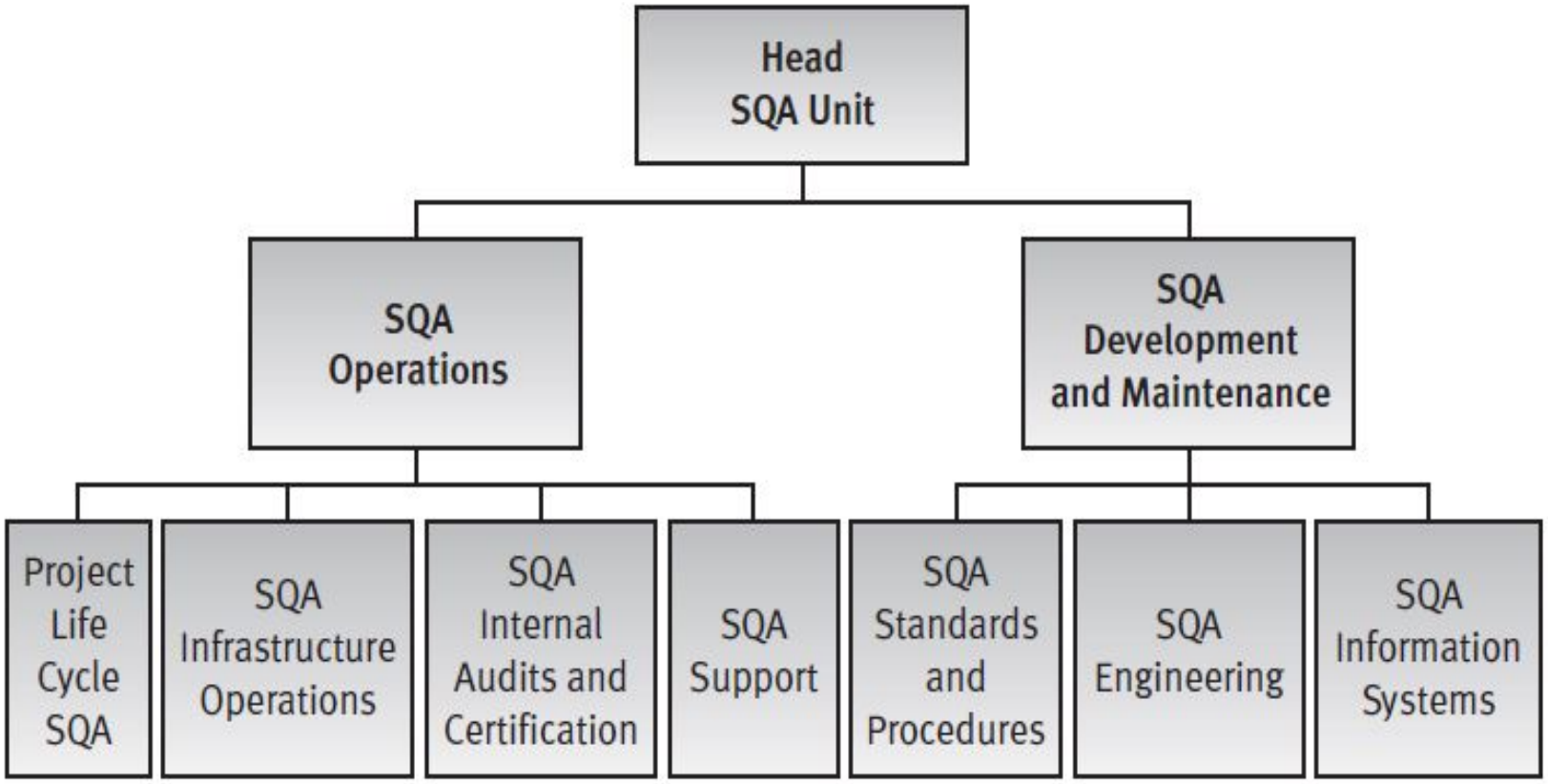

Figure 26.1: Proposed model for an SQA unit's organizational structure

The SQA system – an SQA architecture

- 2. SQA trustees and their tasks

- ***Unit-related tasks***
- Support their colleagues' attempts to solve difficulties arising in the implementation of software quality procedures and work instructions
- Promote compliance and monitor implementation of SQA procedures and work instructions by colleagues
- Report severe software quality failures to the SQA unit.
- ***Organization-related tasks***
- ■ Initiate changes and updates of *organization-wide* SQA procedures and work instructions
- ■ Identify organization-wide SQA training needs and propose an appropriate training or instruction program to be carried out by the SQA unit.

The SQA system – an SQA architecture

- **3. SQA committees and their tasks**

- SQA committees may be **permanent** or **ad hoc**.
- **Permanent committees** are part of the SQA structure and handle ongoing activities such as software change control, corrective actions, procedures, tools, and quality metrics.
- Ad hoc SQA committees handle **temporary, specific issues** such as updating procedures, analyzing and resolving software failures, defining metrics for a particular product or process, and updating quality cost and data collection methods for a given problem.
- Permanent committees operate under defined SQA procedures, while ad hoc committees are created for short-term problems with tasks defined by the authority that forms them.

The SQA system – an SQA architecture

4. SQA forum members

- SQA forums typically focus on:
  - SQA procedure improvements and implementation
  - Quality metrics
  - Corrective actions – analysis of failure and success cases
  - Quality system issues – development and implementation of new tools
- Members of an open forum may include:
  - SQA unit members
  - SQA trustees
  - Software development and maintenance staff
  - SQA and software engineering consultants/experts
  - Customer representatives.